

Week 2 Day 2 — Raw Python vs. LangChain Agent

Raw Python vs. LangChain

In Day 1, the agent was implemented directly in Python, so the model interaction, tool dispatching, execution loop, working memory, and conversation history had to be handled explicitly. In LangChain, these responsibilities are represented through reusable abstractions: the model wrapper provides a standard LLM interface, `@tool` exposes functions with names, schemas and descriptions, `AgentExecutor` manages the tool-calling loop, and memory components preserve conversation history. This reduces boilerplate while keeping the same fundamental agent pattern: the model decides what action is required, a tool performs the action, its result is returned to the model, and the model continues until it can produce a final answer.

LCEL and Tools

The notebook first demonstrates LCEL using the prompt | model | parser pipeline. The pipe operator composes LangChain runnables so that the output of one component becomes the input of the next. Three tools are then registered: calculator, weather_lookup, and lookup_customer. The first two reuse functionality from the raw-Python agent, while lookup_customer reads a local customer_records.json data source. The tool docstrings are important because they become part of the tool metadata presented to the model, helping it understand what each tool does and when it should be used.

Annotated Agent Trace

The LangChain agent is created with `create_tool_calling_agent` and executed using `AgentExecutor` with `verbose=True` and `return_intermediate_steps=True`. For the multi-step test, the user asks the agent to calculate $(12 + 8) / 4$ and retrieve customer C001.

- 1. REASON / DECISION:** The model determines that the request requires both arithmetic and customer information.
- 2. ACT:** The agent calls calculator with $(12 + 8) / 4$.
- 3. OBSERVE:** The tool returns 5.0.
- 4. ACT:** The agent calls lookup_customer for C001.
- 5. OBSERVE:** The tool returns the customer's record, including plan and status.
- 6. FINAL:** The model combines the tool results into the final response.

This trace is structurally similar to the raw-Python agent from Day 1. The major difference is that LangChain hides much of the implementation plumbing: message construction, tool-call parsing, dispatch, and loop management are handled by the

framework. The intermediate trace exposes actions and observations, but not the model's private chain-of-thought.

Memory, Structured Output and Error Handling

The notebook then adds ConversationBufferMemory and tests a three-turn conversation: finding the wireless keyboard price, comparing it with the mechanical keyboard, and choosing one for a budget-conscious client. The memory allows later turns to resolve references such as “it” and “which one” using earlier conversation context. The local product data shows the wireless keyboard at \$45 and the mechanical keyboard at \$95, leading to the wireless keyboard recommendation.

Finally, structured output is introduced through a Pydantic model, while a deliberately flaky inventory tool demonstrates error handling. `handle_tool_error` converts a tool exception into an observation that the agent can respond to, while the prompt instructs the agent to retry and `max_iterations` provides a safety limit. The main limitation observed is that LangChain's abstractions do not remove complexity completely: versioned modules, provider-specific behavior, message formats, and explicit error-handling configuration still have to be understood.

Conclusion

Overall, LangChain makes the raw-Python agent easier to build, extend and maintain by providing standard abstractions for models, tools, agents, memory and structured outputs. However, the framework is not “magic”; understanding the underlying model/tool loop remains important for debugging and for knowing what information is actually visible in a trace. The exercise shows that LangChain mainly removes implementation plumbing while preserving the same core agent architecture developed manually in Day 1.